Problem Statement : Spam or ham?

Import Required Libraries

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer

from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

import warnings
warnings.filterwarnings('ignore')
```

Load Data

In [ ]:
```
spam = pd.read_csv('/content/spam.csv')
spam.head()
```

Out[2]:

	Category	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

In [ ]:
```
spam.tail()
```

Out[3]:

	Category	Message
5567	spam	This is the 2nd time we have tried 2 contact u...
5568	ham	Will ü b going to esplanade fr home?
5569	ham	Pity, * was in mood for that. So...any other s...
5570	ham	The guy did some bitching but I acted like i'd...
5571	ham	Rofl. Its true to its name

In [ ]:
```
spam.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   Category  5572 non-null   object
 1   Message   5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
```

In [ ]:
```
spam.isna().sum()
```

Out[5]:

	0
Category	0
Message	0

dtype: int64

Feature Extraction : CountVectorizer

In [ ]:
```
train_set = {"The sky is blue.", "The sun is bright."}
test_set = {"The sun in the sky is bright.","we can see the shining sun, the bright sun."}
```

In [ ]:
```
vectorizer = CountVectorizer()
```

In [ ]:
```
vectorizer.fit(train_set)
vectorizer.vocabulary_
```

Out[8]:
```
{'the': 5, 'sun': 4, 'is': 2, 'bright': 1, 'sky': 3, 'blue': 0}
```

- keys : unique words in the training sentences
- values : indices based on their appearance order when the vocabulary was built

In [ ]:
```
# we can sort the doctionary
voacb_dict = vectorizer.vocabulary_.copy()
dict(sorted(voacb_dict.items(), key=lambda x: x[1]))
```

Out[9]:
```
{'blue': 0, 'bright': 1, 'is': 2, 'sky': 3, 'sun': 4, 'the': 5}
```

In [ ]:
```
test_set
```

Out[10]:
```
{'The sun in the sky is bright.',
 'we can see the shining sun, the bright sun.'}
```

In [ ]:
```
test_vec = vectorizer.transform(test_set)
print(test_vec)
```

```
<Compressed Sparse Row sparse matrix of dtype 'int64'
        with 8 stored elements and shape (2, 6)>
  Coords        Values
  (0, 1)        1
  (0, 2)        1
  (0, 3)        1
  (0, 4)        1
  (0, 5)        2
  (1, 1)        1
  (1, 4)        2
  (1, 5)        2
```

- Each entry is represented as `(row_index, column_index)` and `count`
- `row_index` : Indicates which document (from test_set) the entry corresponds to.
- `column_index` : Indicates which word (from the training vocabulary) is being counted.
- `count` : Indicates how many times the word appears in that document.

In [ ]:
```
# convert to array
test_vec.toarray()
```

Out[12]:
```
array([[0, 1, 1, 1, 1, 2],
       [0, 1, 0, 0, 2, 2]])
```

In [ ]:
```
# inverse transform
vectorizer.inverse_transform(test_vec)
```

Out[13]:
```
[array(['bright', 'is', 'sky', 'sun', 'the'], dtype='<U6'),
 array(['bright', 'sun', 'the'], dtype='<U6')]
```

We only got the token back, not the oreder of words. This is limitation of coutvectorizer.

Data processing

In [ ]:
```
# Convierting category into binary column
spam['Category']=spam['Category'].apply(lambda x:1 if x=='spam' else 0)
spam.head(5)
```

Out[14]:

	Category	Message
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

In [ ]:
```
X = spam['Message']
y = spam['Category']
```

In [ ]:
```
X.shape, y.shape
```

Out[16]:
```
((5572,), (5572,))
```

In [ ]:
```
X.head()
```

Out[17]:

	Message
0	Go until jurong point, crazy.. Available only ...
1	Ok lar... Joking wif u oni...
2	Free entry in 2 a wkly comp to win FA Cup fina...
3	U dun say so early hor... U c already then say...
4	Nah I don't think he goes to usf, he lives aro...

dtype: object

In [ ]:
```
y.head()
```

Out[18]:

	Category
0	0
1	0
2	1
3	0
4	0

dtype: int64

In [ ]:
```
## apply count vectorizer
vect = CountVectorizer(ngram_range=(1,3),min_df=5,max_features=8000)
X_vect = vect.fit_transform(X).toarray()
X_vect[:5]
```

Out[19]:
```
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]])
```

In [ ]:
```
vect.fit_transform??
```

In [ ]:
```
X_vect.shape
```

Out[21]:
```
(5572, 4722)
```

4000+ Features got created by vectorizer.

In [ ]:
```
# split data
X_train, X_test, y_train, y_test = train_test_split(X_vect, y,
test_size=0.2, random_state=42)
```

In [ ]:
```
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

Out[23]:
```
((4457, 4722), (1115, 4722), (4457,), (1115,))
```

Build Naive Bayes Model

In [ ]:
```
nb = GaussianNB()
nb.fit(X_train, y_train)
```

Out[24]:
```
GaussianNB()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

GaussianNB
?Documentation for GaussianNBiFitted

```
GaussianNB()
```

In [ ]:
```
# training score
nb.score(X_train, y_train)
```

Out[25]:
```
0.9479470495849226
```

In [ ]:
```
# test score
nb.score(X_test, y_test)
```

Out[26]:
```
0.9452914798206278
```

Comparing Naive Bayes with KNN and SVM

In [ ]:
```
# build KNN

%%time
knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
knn.score(X_train, y_train), knn.score(X_test, y_test)
```

```
CPU times: user 9.23 s, sys: 260 ms, total: 9.49 s
Wall time: 6.45 s
```

Out[27]:
```
(0.9360556428090644, 0.9291479820627803)
```

In [ ]:
```
# build SVM

%%time
svm = SVC()
svm.fit(X_train, y_train)
svm.score(X_train, y_train), svm.score(X_test, y_test)
```

```
CPU times: user 1min 15s, sys: 259 ms, total: 1min 15s
Wall time: 1min 21s
```

Out[28]:
```
(0.9943908458604442, 0.9838565022421525)
```

In [ ]:
```
# build Naive Bayes

%%time
nb = GaussianNB()
nb.fit(X_train, y_train)
nb.score(X_train, y_train), nb.score(X_test, y_test)
```

```
CPU times: user 732 ms, sys: 431 ms, total: 1.16 s
Wall time: 1.16 s
```

Out[29]:
```
(0.9479470495849226, 0.9452914798206278)
```

Prediction

In [ ]:
```
sample = "You won 25 lakh"
vec = vect.transform([sample]).toarray()
nb.predict(vec)
```

Out[30]:
```
array([1])
```

Correct Classification

In [ ]:
```
sample = "Hi Rushikesh, I need your help"
vec = vect.transform([sample]).toarray()
nb.predict(vec)
```

Out[31]:
```
array([1])
```

In [ ]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix, classification_report
```

In [ ]:
```
y_pred = nb.predict(X_test)
```

In [ ]:
```
y_test
```

Out[36]:

	Category
3245	0
944	0
1044	0
2484	0
812	0
...	...
4264	0
2439	0
5556	0
4205	0
4293	0

1115 rows × 1 columns

dtype: int64

In [ ]:
```
y_pred
```

Out[35]:
```
array([0, 0, 0, ..., 0, 0, 1])
```

In [ ]:
```
CM = confusion_matrix(y_test, y_pred)
```

In [ ]:
```
CM
```

Out[40]:
```
array([[913,  53],
       [  8, 141]])
```

In [ ]:
```
precision = precision_score(y_pred, y_test)
```

In [ ]:
```
precision
```

Out[42]:
```
0.9463087248322147
```

In [ ]:
```
recall = recall_score(y_pred, y_test)
```

In [ ]:
```
recall
```

Out[44]:
```
0.7268041237113402
```

In [ ]:
```
f1 = f1_score(y_pred, y_test)
```

In [ ]:
```
f1
```

Out[46]:
```
0.8221574344023324
```

In [ ]:
```
cr = classification_report(y_pred, y_test)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_1899/3974941403.py in <cell line: 0>()
----> 1 cr = classification_report(y_pred, y_test)

TypeError: 'str' object is not callable
```

In [ ]:
```
classification_report
```

Out[49]:
```
'              precision    recall  f1-score   support\n\n           0
0.95      0.99      0.97       921\n           1       0.95      0.73
0.82       194\n\n    accuracy                           0.95      1115\n
macro avg       0.95      0.86      0.89      1115\nweighted avg       0.95
0.95      0.94      1115\n'
```

In [ ]:
```
              precision    recall  f1-score   support

           0       0.95      0.99      0.97       921
           1       0.95      0.73      0.82       194

    accuracy                           0.95      1115
   macro avg       0.95      0.86      0.89      1115
weighted avg       0.95      0.95      0.94      1115
```

```
          precision    recall  f1-score   support

       0       0.95      0.99      0.97       921
       1       0.95      0.73      0.82       194

accuracy                           0.95      1115
```

macro avg 0.95 0.86 0.89 1115 weighted avg 0.95 0.95 0.94 1115